RV1106 RV1103 快速入门

文件标识：RK-JC-YF-415

发布版本：V2.5.0

日期：2026-09-30

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档面向首次使用 RV1103B/RV1103/RV1106B/RV1106 RKNN SDK 的用户，目标是在最短路径内完成开发板连接、RKNN-Toolkit2 环境准备、模型转换、板端运行和首次推理验证。本文所用示例已集成到 RKNN Model Zoo 中。

读者对象

本文档主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本	修改人	修改日期	修改说明	核定人
V1.6.0	HPC	2023-11-28	初始版本	熊伟
V2.0.0-beta0	HPC	2024-03-15	更新版本号	熊伟
V2.1.0	HPC	2024-08-01	增加 RV1103B 支持	熊伟
V2.2.0	HPC	2024-09-04	增加 RV1106B 支持	熊伟
V2.3.0	HPC	2024-11-06	增加通过 pip 源安装 RKNN-Toolkit2 的支持	熊伟
V2.3.2	HPC	2025-04-03	更新版本号	熊伟
V2.4.2	HPC	2026-03-19	更新版本号	熊伟
V2.5.0	HPC	2026-09-27	文档结构重构	熊伟

目录

RV1106 RV1103 快速入门

1. 开始之前

    1.1 本指南目标

    1.2 了解 RKNN SDK 部署链路

    1.3 支持平台和版本要求

    1.4 选择示例类型

    1.5 获取 SDK 及配套仓库

2. 硬件连接与检查

    2.1 硬件清单

    2.2 开发板和连接工具介绍

    2.3 连接开发板

    2.4 检查设备连接状态

3. 主机端和板端环境准备

    3.1 安装 RKNN-Toolkit2

        3.1.1 安装 Python 环境

        3.1.2 安装 RKNN-Toolkit2

    3.2 检查 Toolkit 安装结果

    3.3 安装或检查板端 RKNPU2 环境

        3.3.1 检查 RKNPU2 驱动版本

        3.3.2 检查 RKNPU2 Mini Runtime 和 Server

        3.3.3 安装或更新 RKNPU2 环境

    3.4 安装编译工具

        3.4.1 安装 CMake

        3.4.2 安装编译器

            3.4.2.1 确认开发板系统架构

            3.4.2.2 安装 GCC 交叉编译器

    3.5 运行环境预检查

4. 运行第一个通用模型

    4.1 获取示例和模型

    4.2 转换模型

    4.3 运行 RKNN Python Demo

    4.4 编译并部署 RKNN C Demo

        4.4.1 编译

        4.4.2 推送文件到板端

4.5 板端运行并验证输出
4.6 数据集精度评估（可选）
5. Docker 中运行 RKNN Python Demo（可选）
5.1 安装 Docker
5.2 准备 RKNN-Toolkit2 Docker 镜像
5.2.1 通过 Dockerfile 构建镜像
5.2.2 加载已打包镜像
5.2.3 检查镜像
5.3 在容器中运行 RKNN Python Demo
6. 常见启动问题与下一步
6.1 设备无法识别
6.2 rknn_server 未正常启动
6.3 内存不足导致模型加载失败
6.4 后续文档导航

1. 开始之前

本指南以最短路径引导用户完成 RV1103/RV1103B/RV1106/RV1106B 上 RKNN SDK 的首次运行，涵盖硬件连接、环境检查、模型转换、连板推理和 C/C++ 板端部署等必要步骤。

> **说明：** 完整开发流程、模型转换与量化、环境安装及系统化故障排查，请参考《RKNN2_SDK_开发指南》；RKNN-Toolkit2 Python 接口请参考《RKNN2_Toolkit_Python_API_参考》；RKNN-Toolkit-Lite2 Python 接口请参考《RKNN2_Toolkit_Lite_Python_API_参考》；板端 C API 及 RKNPU2 Mini Runtime（`librknnmrt.so`）相关接口请参考《RKNN2_Runtime_C_API_参考》。

1.1 本指南目标

完成本指南后，用户应能够：

1. 确认开发主机与开发板连接正常；
2. 安装并验证 RKNN-Toolkit2 开发环境；
3. 检查或更新板端 RKNPU2 Mini Runtime（`librknnmrt.so`）和 `rknn_server`；
4. 使用 RKNN Model Zoo 完成 YOLOv5 模型下载与 RKNN 模型转换；
5. 通过 RKNN Python Demo 完成一次连板推理；
6. 编译并在板端运行 RKNN C Demo，验证输出结果。

1.2 了解 RKNN SDK 部署链路

RV1103/RV1103B/RV1106/RV1106B 的首次运行主要涉及计算机端开发工具和开发板端运行环境。

组件	所在位置	作用
RKNN-Toolkit2	计算机端	将 ONNX 等原始模型转换为 RKNN 模型，并支持连板调试
RKNN Model Zoo	计算机端	提供模型转换、Python Demo 和 C/C++ Demo 示例
`rknn_server`	开发板端	处理计算机通过 USB 发送的连板推理请求
RKNPU2 Mini Runtime（`librknnmrt.so`）	开发板端	加载 RKNN 模型并调用 NPU 执行推理

RKNPU2 Mini Runtime 与 RKNPU2 Runtime 的区别如下：

对比项	RKNPU2 Mini Runtime	RKNPU2 Runtime
运行库	`librknnmrt.so`	`librknnrt.so`
定位	面向 RV1103/RV1103B/RV1106/RV1106B 等资源受限平台的裁剪版 Runtime	RKNPU2 标准 Runtime
本文适用性	**适用，本文后续板端运行库均指 `librknnmrt.so`**	**不适用于本文示例，不应使用 `librknnrt.so` 替换 `librknnmrt.so`**

> **注意：** 本文后续统一使用 **RKNPU2 Mini Runtime** 表示板端 `librknnmrt.so`。除本节用于说明两种 Runtime 的区别外，不再使用“RKNPU2 Runtime”泛称板端运行库，以避免与 `librknnrt.so` 混淆。

首次运行的推荐路径为：

1. 连接开发板并确认 `adb devices` 可识别设备；
2. 安装 RKNN-Toolkit2 并完成导入检查；
3. 检查 RKNPU2 驱动、Mini Runtime 和 `rknn_server`；
4. 下载 YOLOv5 示例模型并转换为 RKNN 格式；
5. 先运行 RKNN Python Demo 验证转换和连板链路；
6. 再编译、推送并运行 RKNN C Demo，完成板端部署验证。

1.3 支持平台和版本要求

本指南支持以下平台：

平台	说明
RV1103	支持
RV1103B	支持
RV1106	支持
RV1106B	支持

开发主机示例基于 Ubuntu 18.04 / Ubuntu 20.04 / Ubuntu 22.04。RKNN-Toolkit2 环境推荐使用 Python 3.8。

> **注意：** RV1103、RV1106 与 RV1103B、RV1106B 生成的 RKNN 模型不能共用。模型转换时应根据实际开发板选择正确的平台参数。

1.4 选择示例类型

本指南使用 RKNN Model Zoo 中的 YOLOv5 示例贯穿首次运行流程。

示例	用途	Model Zoo 目录
YOLOv5 Python Demo	模型转换与连板推理验证	`examples/yolov5/python`
YOLOv5 C Demo	C/C++ 编译与板端部署验证	`examples/yolov5/cpp`

Python Demo 和 C Demo 共用同一份 YOLOv5 ONNX 模型及转换后的 RKNN 模型，因此只需要完成一次模型下载和转换。

1.5 获取 SDK 及配套仓库

建议创建专门目录存放 RKNN 相关仓库，例如创建名为 `Projects` 的文件夹，并将 RKNN-Toolkit2 和 RKNN Model Zoo 存放在该目录下。

```
# 新建 Projects 文件夹
mkdir Projects

# 进入该目录
cd Projects

# 下载 RKNN-Toolkit2 仓库
git clone https://github.com/airockchip/rknn-toolkit2.git --depth 1

# 下载 RKNN Model Zoo 仓库
git clone https://github.com/airockchip/rknn_model_zoo.git --depth 1
```

> **说明：** `--depth 1` 表示只克隆最近一次 commit。如果 `git clone` 失败，也可以直接在 GitHub 中下载压缩包并解压到该目录。

后续示例默认目录结构如下：

```
Projects
├── rknn-toolkit2
│   ├── doc
│   ├── rknn-toolkit2
│   │   ├── packages
│   │   ├── docker
│   │   └── ...
│   ├── rknpu2
│   │   ├── runtime
│   │   └── ...
│   └── ...
└── rknn_model_zoo
    ├── datasets
    ├── examples
    └── ...
```

2. 硬件连接与检查

本章只完成首次运行所需的硬件连接和设备可见性检查。确认开发板能够通过 ADB 被计算机识别后，再继续环境安装和模型转换。

2.1 硬件清单

运行本文档示例程序需要准备以下硬件：

硬件	数量
Ubuntu 计算机	1
RV1103/RV1103B/RV1106/RV1106B 开发板	1
Micro USB 数据线	1
12V-2A 电源适配器	1

2.2 开发板和连接工具介绍

RV1103 开发板

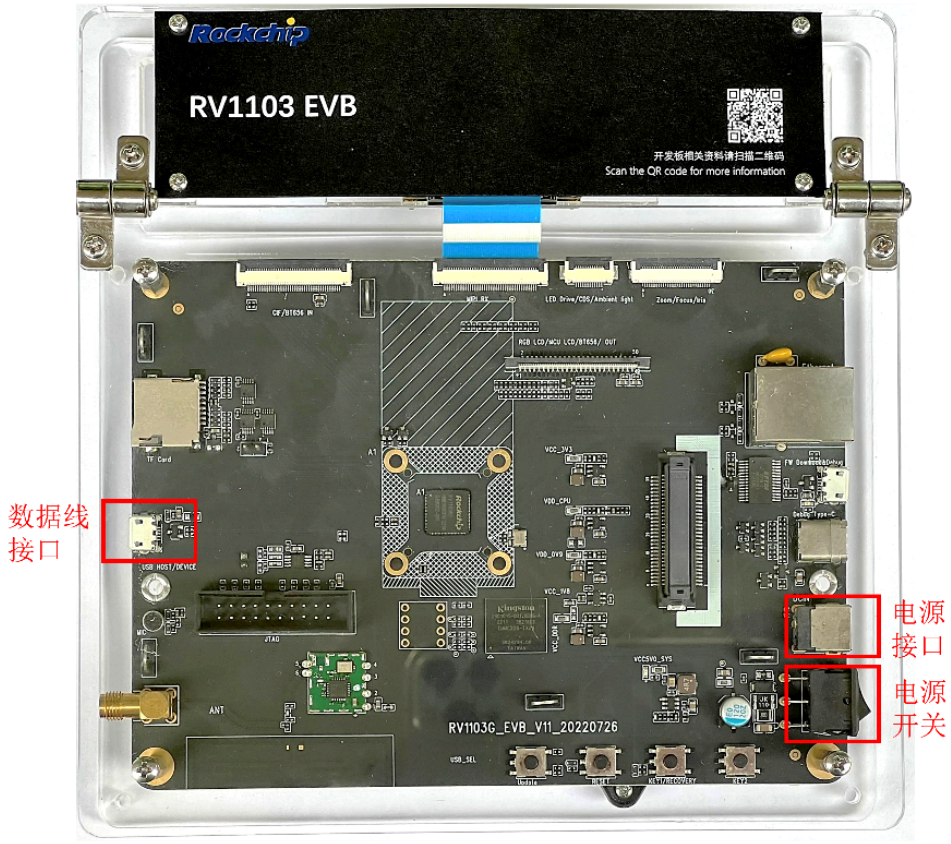

图2-1 RV1103开发板

RV1103B 开发板

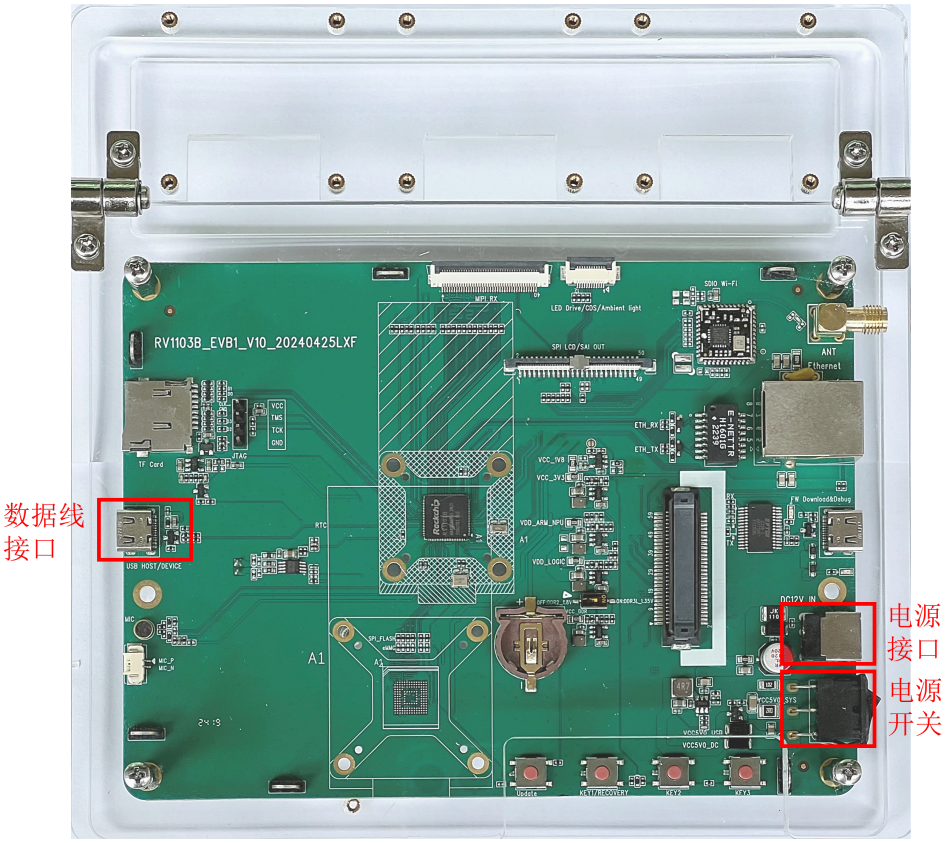

图2-2 RV1103B开发板

RV1106 开发板

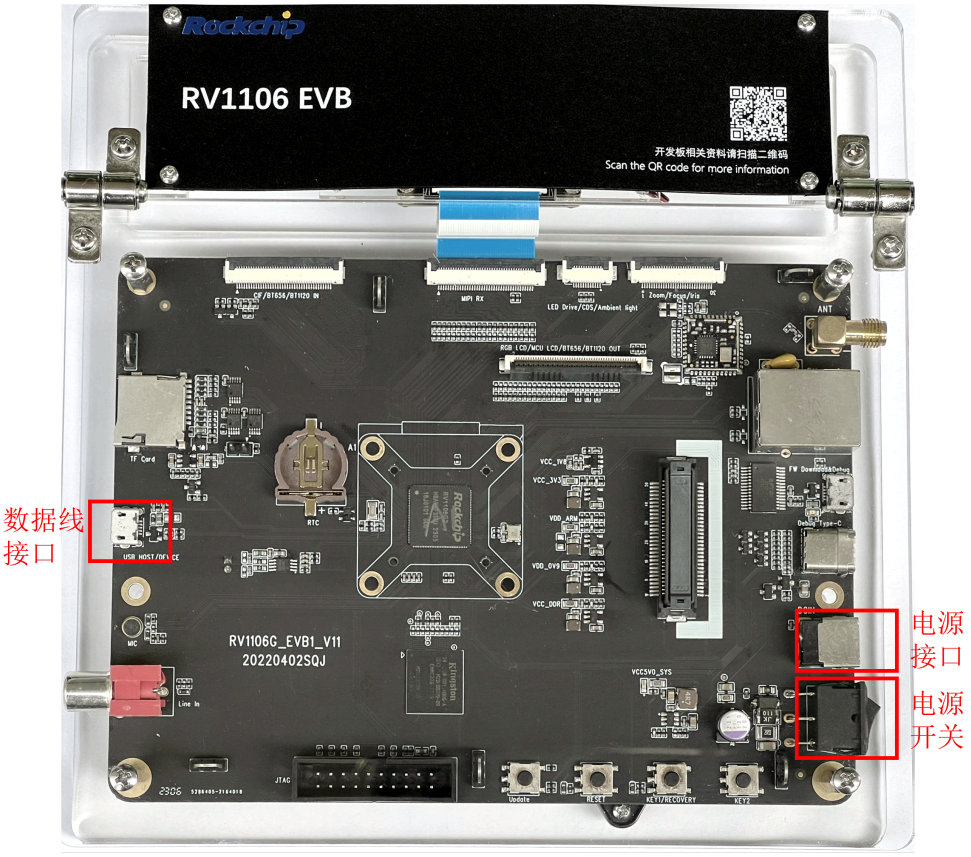

图2-3 RV1106开发板

Micro USB 数据线

图2-4 Micro USB 数据线

电源适配器

图2-5 输出 12V-2A 的电源适配器

2.3 连接开发板

下面以 RV1106 为例说明如何将开发板连接到计算机：

1. 准备一台操作系统为 Ubuntu 18.04 / Ubuntu 20.04 / Ubuntu 22.04 的计算机；
2. 找到开发板电源接口并连接电源适配器；
3. 使用数据线连接开发板与计算机。由于生产批次不同，开发板的数据线接口类型和位置可能发生变化，通常开发板上印有 `OTG` 字样的接口为数据线接口；
4. 打开电源开关，等待开发板系统启动完成。

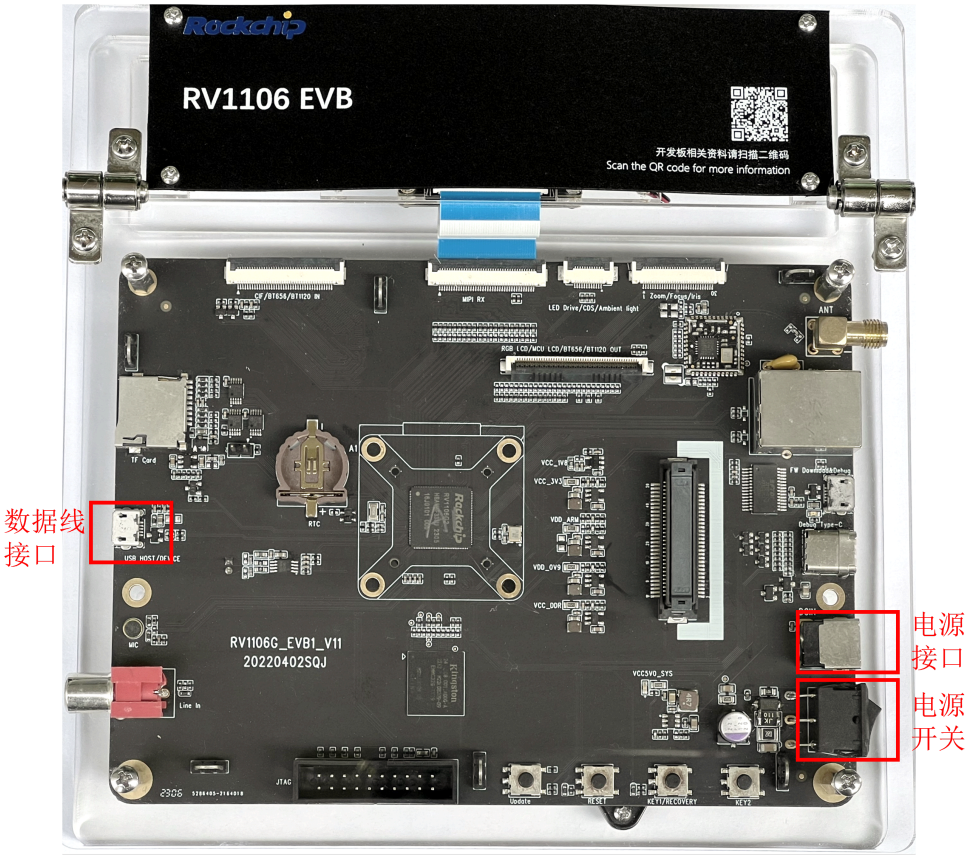

图2-6 RV1106开发板

2.4 检查设备连接状态

在计算机终端中执行：

```
# 如果没有安装 adb，请先安装
sudo apt install adb

# 查询 ADB 设备
adb devices
```

连接成功时输出信息如下，其中 `421ca310c9b9242b` 为 RV1106 的设备 ID（示例值，实际以本机为准）：

```
List of devices attached
421ca310c9b9242b    device
```

在继续安装和模型转换前，至少确认以下检查通过：

检查项	检查命令	预期结果
ADB 工具可用	`adb version`	能正常输出 ADB 版本信息
开发板可见	`adb devices`	列表中出现状态为 `device` 的开发板

如果 `adb devices` 无设备显示，请参考 6.1 设备无法识别。

3. 主机端和板端环境准备

本章完成首次运行需要的 RKNN-Toolkit2、RKNPU2 和编译工具准备。后续主流程以计算机中直接安装环境为例；如需使用 Docker，可参考第 5 章。

3.1 安装 RKNN-Toolkit2

3.1.1 安装 Python 环境

如果系统中没有安装 Python 3.8（建议版本），或者同时存在多个 Python 环境，建议使用 Miniforge 创建独立的 Python 3.8 环境。

步骤 1：安装 Miniforge

首先检查系统是否已经安装 Miniforge；若已安装，可跳过本步骤。

```
conda -V
# 参考输出：conda 23.3.1，表示 Miniforge conda 版本为 23.3.1
# 如果提示 conda: command not found，则表示未安装 Miniforge
```

如果未安装 Miniforge，可下载安装包：

```
wget -c https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh
```

然后执行：

```
chmod +x Miniforge3-Linux-x86_64.sh
bash Miniforge3-Linux-x86_64.sh
```

步骤 2：创建 Python 环境

进入 Miniforge base 环境：

```
source ~/miniforge3/bin/activate
# 成功后，命令行提示符会变成以下形式：
# (base) xxx@xxx:~$
```

创建名为 `toolkit2` 的 Python 3.8 环境：

```
conda create -n toolkit2 python=3.8
```

激活 `toolkit2` 环境：

```
conda activate toolkit2
# 成功后，命令行提示符会变成以下形式：
# (toolkit2) xxx@xxx:~$
```

> **说明：** 后续 Python 相关命令默认在 `toolkit2` 环境中执行。

3.1.2 安装 RKNN-Toolkit2

激活 `toolkit2` 环境后，可通过 pip 源或本地 wheel 包安装 RKNN-Toolkit2。

方式 1：通过 pip 源安装

```
pip install rknn-toolkit2 -i https://pypi.org/simple

# 如果已安装 RKNN-Toolkit2，可通过以下命令升级
pip install rknn-toolkit2 -i https://pypi.org/simple --upgrade
```

方式 2：通过本地 wheel 包安装

```
# 进入 rknn-toolkit2 目录
cd Projects/rknn-toolkit2/rknn-toolkit2

# 根据 Python 版本及处理器架构选择 requirements 文件
# cpxx 为 Python 版本号
pip install -r packages/x86_64/requirements_cpxx.txt
# pip install -r packages/arm64/arm64_requirements_cpxx.txt

# 安装 RKNN-Toolkit2
# x.x.x 为 RKNN-Toolkit2 版本号，cpxx 为 Python 版本号
pip install packages/x86_64/rknn_toolkit2-x.x.x-cpxx-cpxx-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl
# pip install packages/arm64/rknn_toolkit2-x.x.x-cpxx-cpxx-
manylinux_2_17_aarch64.manylinux2014_aarch64.whl
```

3.2 检查 Toolkit 安装结果

保持 `toolkit2` 环境处于激活状态，先进入 Python 交互模式：

```
python
```

然后导入 `RKNN` 类：

```
from rknn.api import RKNN
```

若导入过程没有报错，表示最基本的 RKNN-Toolkit2 安装检查通过。

如果安装失败，请查阅《RKNN2_SDK_开发指南》中 RKNN-Toolkit2 安装与故障排查相关章节。

3.3 安装或检查板端 RKNPU2 环境

为便于描述，后续文档使用“板端”表示开发板端。

RKNN-Toolkit2 的连板调试功能要求板端已安装 RKNPU2 环境并启动 `rknn_server`。其中：

组件	作用
RKNN Server（`rknn_server`）	运行在开发板上的后台代理服务，处理计算机通过 USB 发送的数据，并调用板端 Mini Runtime 完成推理
RKNPU2 Mini Runtime（`librknnmrt.so`）	在系统中加载 RKNN 模型并调用 NPU 执行推理

3.3.1 检查 RKNPU2 驱动版本

在板端执行以下命令查询 RKNPU2 驱动版本：

```
dmesg | grep -i rknpu
```

如下图所示，示例中的 RKNPU2 驱动版本为 0.8.5：

```
# dmesg | grep -i rknpu
[    4.431371] RKNPU ff660000.npu: RKNPU: rknpu iommu device-tree entry not found!, using non-iommu mode
[    4.432210] RKNPU ff660000.npu: RKNPU: Initialized RKNPU driver: v0.8.5 for 20230202
[    4.432314] RKNPU ff660000.npu: dev_pm_opp_set_regulators: no regulator (rknpu) found: -19
```

图3-1 RKNPU2 驱动版本信息

Rockchip 开发板官方固件均自带 RKNPU2 驱动。若以上命令查询不到 NPU 驱动版本，则可能使用了未集成 NPU 驱动的第三方固件。如果有固件源码，可以在 kernel config 中将 `CONFIG_ROCKCHIP_RKNPU` 设置为 `y`，重新编译内核驱动并烧录。

> **建议：** RKNPU2 驱动版本 >= 0.9.2。

3.3.2 检查 RKNPU2 Mini Runtime 和 Server

如果板端没有安装 RKNN Server 和 Mini Runtime 库，或两者版本不一致，需要重新安装 RKNPU2 环境。

> **注意：**
>
> 1. 如果使用动态维度输入的 RKNN 模型，RKNN Server 和 Mini Runtime 库版本需要 >= 1.5.0；
> 2. `rknn_server`、Mini Runtime 库和 RKNN-Toolkit2 的版本应保持一致，建议安装同一套最新版本。

步骤 1：检查 `rknn_server` 是否可启动

```
# 进入板端
adb shell

# 启动 rknn_server
restart_rknn.sh
```

如果出现类似以下输出，表示 `rknn_server` 启动成功：

```
start rknn server, version: x.x.x
```

步骤 2：检查 Server 和 Mini Runtime 版本

```
# 查询 rknn_server 版本
strings /oem/usr/bin/rknn_server | grep -i "rknn_server version"

# 查询 librknnmrt.so 版本
strings /oem/usr/lib/librknnmrt.so | grep -i "librknnmrt version"
```

参考输出：

```
rknn_server version: x.x.x
librknnmrt version: x.x.x
```

3.3.3 安装或更新 RKNPU2 环境

如果已经安装版本一致的 RKNPU2 环境，可跳过本节。

进入 `rknpu2` 目录，使用 ADB 将对应的 `rknn_server` 和 `librknnmrt.so` 推送至板端，然后启动 `rknn_server`：

```
# 进入 rknpu2 目录
cd Projects/rknn-toolkit2/rknpu2

# 推送 rknn_server 到板端
adb push runtime/Linux/rknn_server/armhf-uclibc/usr/bin/* /oem/usr/bin

# 推送 librknnmrt.so
adb push runtime/Linux/librknn_api/armhf-uclibc/librknnmrt.so /oem/usr/lib

# 进入板端
adb shell

# 赋予可执行权限
chmod +x /oem/usr/bin/rknn_server
chmod +x /oem/usr/bin/start_rknn.sh
chmod +x /oem/usr/bin/restart_rknn.sh

# 重启 rknn_server 服务
restart_rknn.sh
```

3.4 安装编译工具

3.4.1 安装 CMake

在计算机终端中执行：

```
# 更新包列表
sudo apt update

# 安装 cmake
sudo apt install cmake
```

3.4.2 安装编译器

3.4.2.1 确认开发板系统架构

在计算机端执行：

```
adb shell uname -a
```

参考输出如下，其中 `armv7l` 表示基于 ARMv7 架构，属于 armhf 架构。该架构类型将在后续编译 RKNN C Demo 时使用。

```
Linux Rockchip 5.10.160 #1 Tue Oct 24 18:52:11 CST 2023 armv7l GNU/Linux
```

3.4.2.2 安装 GCC 交叉编译器

GCC 下载地址：https://console.zbox.filez.com/l/c1cKaR（提取码：`rknn`）。

下载后解压软件包。建议将 GCC 软件包解压到 `Projects` 目录：

```
Projects
├── rknn-toolkit2
├── rknn_model_zoo
└── arm-rockchip831-linux-uclibcgnueabihf
```

后续示例使用的 GCC 编译器路径为：

```
Projects/arm-rockchip831-linux-uclibcgnueabihf/bin/arm-rockchip831-linux-
uclibcgnueabihf
```

3.5 运行环境预检查

开始模型转换前，建议完成以下检查：

检查项	命令或位置	预期结果
Toolkit	按第 3.2 节进入 Python 并执行 `from rknn.api import RKNN`	无报错
Model Zoo	`Projects/rknn_model_zoo/examples/yolov5`	示例目录存在
ADB	`adb devices`	开发板状态为 `device`
RKNPU2 驱动	`adb shell "dmesg | grep -i rknpu"`	能查询到 RKNPU2 驱动信息
RKNN Server	`adb shell restart_rknn.sh`	能正常启动 `rknn_server`
Mini Runtime 版本	查询 `rknn_server` 和 `librknnmrt.so` 版本	版本一致
CMake	`cmake --version`	能正常输出版本信息
GCC	检查 `Projects/arm-rockchip831-linux-uclibcgnueabihf`	交叉编译器目录存在

上述检查通过后，再继续运行第一个模型。

4. 运行第一个通用模型

本章以 RV1106 平台和 YOLOv5 为例，完成模型获取、RKNN 模型转换、Python 连板推理和 C/C++ 板端部署。

> **说明：** 不同 Model Zoo 示例的参数和用法可能存在差异，请以对应示例目录下的 `README.md` 为准。

4.1 获取示例和模型

RKNN Model Zoo 提供常用模型的示例代码，工程目录结构如下：

```
rknn_model_zoo
├── 3rdparty           # 第三方库
├── datasets           # 数据集
├── examples           # 示例代码
├── utils              # 常用方法，如文件操作、画图等
├── build-android.sh   # Android 开发板编译脚本
├── build-linux.sh     # Linux 开发板编译脚本
└── ...
```

以 YOLOv5 为例：

```
rknn_model_zoo
├── examples
│   └── yolov5
│       ├── cpp        # C/C++ 版本示例代码
│       ├── model      # 模型、测试图片等文件
│       ├── python     # 模型转换脚本和 Python 版本示例代码
│       └── README.md
└── ...
```

进入 `model` 目录并运行 `download_model.sh`，下载 YOLOv5 ONNX 模型：

```
cd Projects/rknn_model_zoo/examples/yolov5/model

# 下载 yolov5 ONNX 模型
# 示例下载结果：model/yolov5s_relu.onnx
./download_model.sh
```

4.2 转换模型

进入 YOLOv5 Python 示例目录，使用 `convert.py` 将 ONNX 模型转换为 RKNN 模型：

```
cd Projects/rknn_model_zoo/examples/yolov5/python

# 用法: python convert.py model_path [rv1103|rv1103b|rv1106|rv1106b] [i8/fp]
[output_path]
python convert.py ../model/yolov5s_relu.onnx rv1106 i8 ../model/yolov5s_relu.rknn
```

> **注意：** RV1103、RV1106 与 RV1103B、RV1106B 生成的模型不能共用。使用其他开发板时，需要修改命令中的平台参数。

4.3 运行 RKNN Python Demo

RKNN Python Demo 可通过连板调试方式在板端运行转换后的 YOLOv5 RKNN 模型：

```
cd Projects/rknn_model_zoo/examples/yolov5/python

# 用法: python yolov5.py --model_path {rknn_model} --target {target_platform} --
img_show
# --img_show 用于显示结果图片
python yolov5.py --model_path ../model/yolov5s_relu.rknn --target rv1106 --img_show
```

如果需要先在计算机端运行原始 ONNX 模型，可执行：

```
python yolov5.py --model_path ../model/yolov5s_relu.onnx --img_show
```

默认输入图片为 `model/bus.jpg`，输出结果如下：

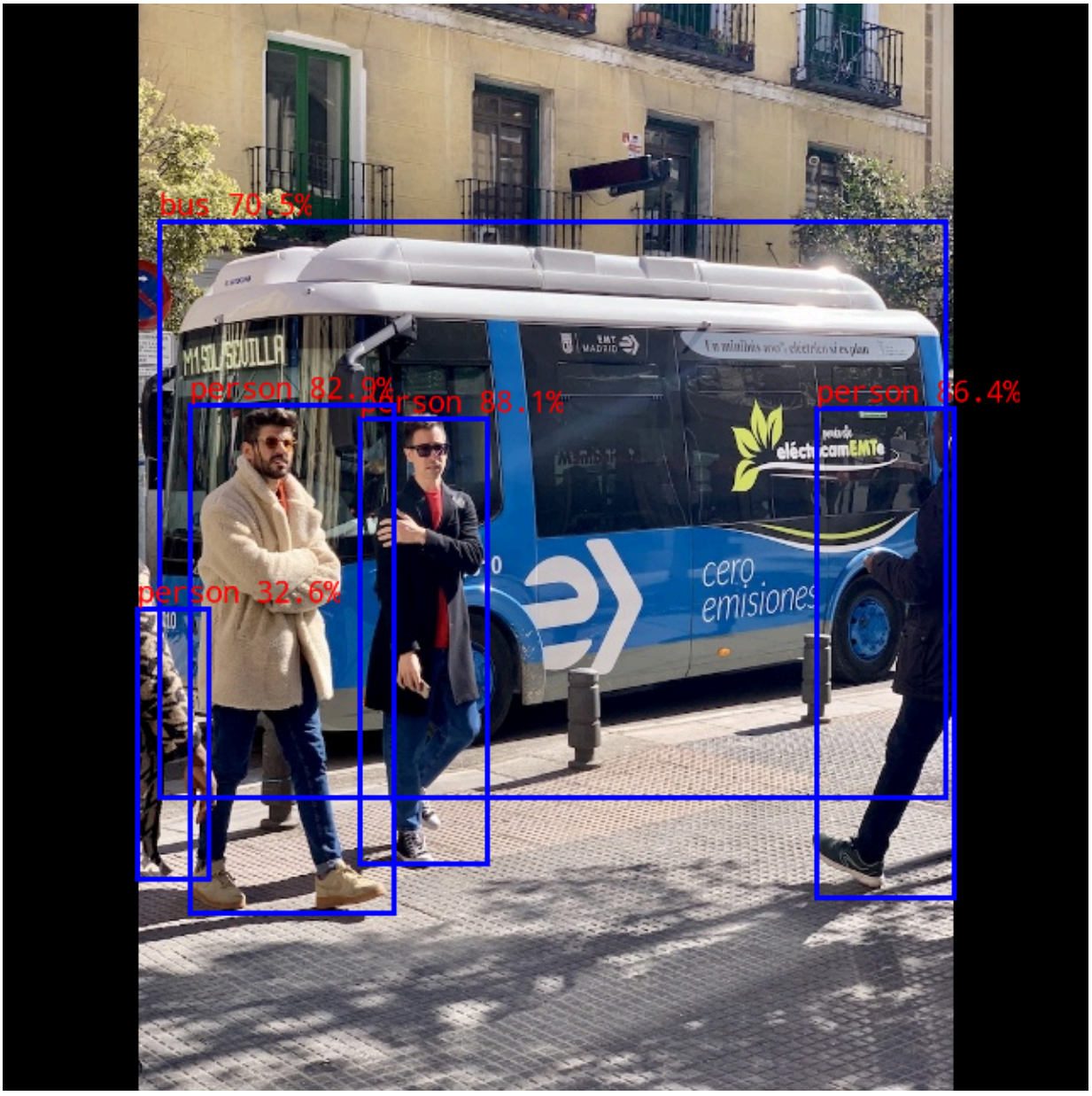

图4-1 RKNN Python Demo 输出图片

如果 Python Demo 能正常完成推理，说明模型转换、ADB 连接、`rknn_server` 和板端 Mini Runtime 的基本链路已经跑通。

4.4 编译并部署 RKNN C Demo

Python Demo 和 C Demo 使用同一份已转换的 RKNN 模型，因此无需重复执行 4.1 和 4.2。

完整运行 RKNN C Demo，需要将 C/C++ 源代码编译为可执行文件，再将可执行文件、模型、测试图片和依赖库推送到板端。

4.4.1 编译

以 Linux 系统、armhf 架构的 RV1106 为例。RV1103/RV1103B 同样为 Linux 系统开发板。

在运行 `build-linux.sh` 前，需要指定本地 GCC 交叉编译器路径。在 `build-linux.sh` 脚本开头加入：

```
GCC_COMPILER=Projects/arm-rockchip831-linux-uclibcgnueabihf/bin/arm-rockchip831-
linux-uclibcgnueabihf
```

然后执行编译：

```
cd Projects/rknn_model_zoo

# 用法: ./build-linux.sh -t <target> -a <arch> -d <build_demo_name> [-b
<build_type>] [-m]
# -t : target (rv1103/rv1106), rv1103b、rv1106b 和 rv1103、rv1106 除模型外其他代码可共用
# -a : arch (aarch64/armhf)
# -d : demo name，对应 examples 目录下子文件夹名称，如 yolov5、mobilenet
# -b : build_type(Debug/Release)
# -m : enable address sanitizer，build_type 需设置为 Debug
./build-linux.sh -t rv1106 -a armhf -d yolov5
```

4.4.2 推送文件到板端

编译完成后，会在 `rknn_model_zoo` 目录下生成 `install` 文件夹，其中包含可执行文件、依赖库、模型和测试图片。参考目录结构：

```
install
└── rv1106_linux_armhf
    └── rknn_yolov5_demo
        ├── lib
        ├── model
        └── rknn_yolov5_demo
```

将整个 Demo 目录推送到板端：

```
cd Projects/rknn_model_zoo

# 使用不同模型和平台时，请在 install 目录下选择对应路径
adb push install/rv1106_linux_armhf/rknn_yolov5_demo /data/
```

4.5 板端运行并验证输出

进入板端并运行可执行文件：

```
adb shell

cd /data/rknn_yolov5_demo/

# 设置依赖库环境
export LD_LIBRARY_PATH=/data/rknn_yolov5_demo/lib

# 用法: ./rknn_yolov5_demo <model_path> <input_path>
./rknn_yolov5_demo model/yolov5s_relu.rknn model/bus.jpg
```

默认输出图片保存为 `rknn_yolov5_demo/out.png`。在计算机端执行以下命令拉取结果：

```
adb pull /data/rknn_yolov5_demo/out.png .
```

输出图片如下：

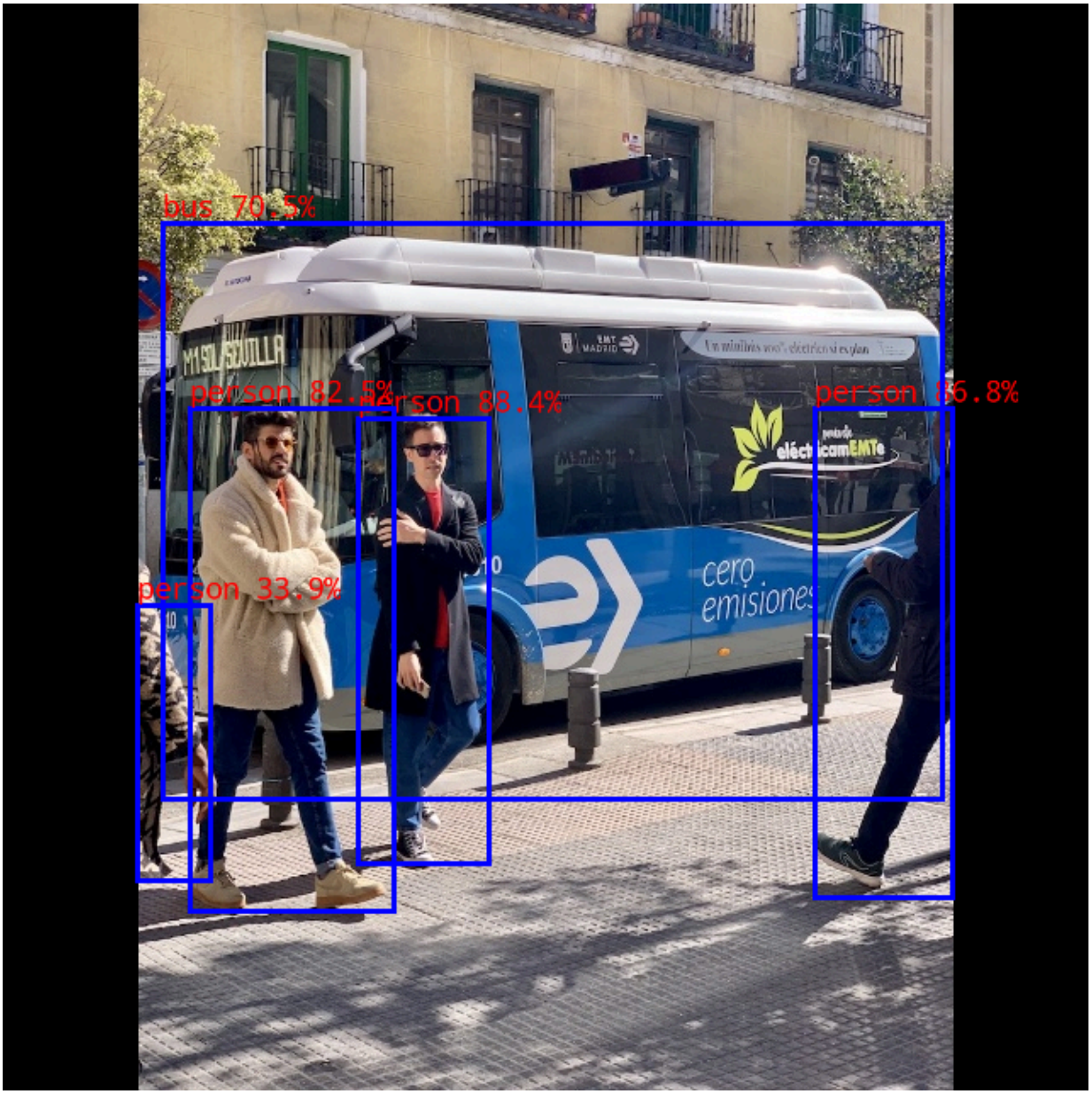

图4-2 RKNN C Demo 输出图片

如果能够正常生成并拉取 `out.png`，说明 C/C++ 编译、文件部署、Mini Runtime 加载和板端推理链路已经完成首次验证。

4.6 数据集精度评估（可选）

`rknn_model_zoo/datasets` 目录用于存放精度评估数据集。对于 YOLOv5，需要先下载 COCO 验证集。

```
cd Projects/rknn_model_zoo/datasets/COCO

# 下载 val2017 数据集
python download_eval_dataset.py
```

进行数据集精度评估时，需要安装 `pycocotools`，并通过 `--coco_map_test` 和 `--img_folder` 指定评估方式及数据集路径：

```
pip install pycocotools

cd Projects/rknn_model_zoo/examples/yolov5/python

python yolov5.py \
    --model_path ../model/yolov5s_relu.rknn \
    --target rv1106 \
    --img_folder ../../../datasets/COCO/val2017 \
    --coco_map_test
```

5. Docker 中运行 RKNN Python Demo（可选）

如果不希望直接在计算机中安装 RKNN-Toolkit2 环境，可以使用包含 RKNN-Toolkit2 环境的 Docker 镜像运行 RKNN Python Demo。

> **注意：** 本章 Docker 镜像只包含 RKNN-Toolkit2 环境，主要用于运行 RKNN Python Demo。板端仍需按照第 3.3 节准备 RKNPU2 环境。

5.1 安装 Docker

如果已经安装 Docker，可跳过本节。未安装时，请参考 Docker 官方安装手册：https://docs.docker.com/install/linux/docker-ce/ubuntu/。

安装完成后，需要将当前用户加入 `docker` 用户组：

```
# 创建 docker 用户组
sudo groupadd docker

# 把当前用户加入 docker 用户组
sudo usermod -aG docker $USER

# 更新激活 docker 用户组
newgrp docker

# 验证无需 sudo 即可执行 docker 命令
docker run hello-world
```

成功时会看到类似以下输出：

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
719385e32844: Pull complete
Digest: sha256:88ec0acaa3ec199d3b7eaf73588f4518c25f9d34f58ce9a0df68429c5af48e8d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
```

5.2 准备 RKNN-Toolkit2 Docker 镜像

可以通过 Dockerfile 构建镜像，也可以加载已经打包的 Docker 镜像文件。

5.2.1 通过 Dockerfile 构建镜像

RKNN-Toolkit2 工程的 `docker/docker_file` 目录提供 Dockerfile：

```
# xx 和 x.x.x 代表版本号，请按实际值替换
cd Projects/rknn-toolkit2/rknn-toolkit2/docker/docker_file/ubuntu_xx_xx_cpxx

docker build -f Dockerfile_ubuntu_xx_xx_for_cpxx -t rknn-toolkit2:x.x.x-cpxx .
```

5.2.2 加载已打包镜像

Docker 镜像文件下载地址：https://console.zbox.filez.com/l/I00fc3（提取码：`rknn`）。

执行：

```
# x.x.x 为 RKNN-Toolkit2 版本号，cpxx 为 Python 版本

docker load --input rknn-toolkit2-x.x.x-cpxx-docker.tar.gz
```

5.2.3 检查镜像

```
docker images
```

参考输出：

```
REPOSITORY       TAG           IMAGE ID       CREATED       SIZE
rknn-toolkit2    x.x.x-cpxx    xxxxxxxxxxxx   1 hours ago   5.89GB
```

5.3 在容器中运行 RKNN Python Demo

首先按照第 1.5 节在本地准备 RKNN Model Zoo，然后将目录和 USB 设备映射到容器中：

```
# -v <host src folder>:<image dst folder> 用于映射本地目录

docker run -t -i --privileged \
    -v /dev/bus/usb:/dev/bus/usb \
    -v "$HOME/Projects/rknn_model_zoo:/rknn_model_zoo" \
    rknn-toolkit2:x.x.x-cpxx \
    /bin/bash
```

进入容器后，准备模型：

```
cd /rknn_model_zoo/examples/yolov5/model
./download_model.sh
```

转换模型：

```
cd /rknn_model_zoo/examples/yolov5/python
python convert.py ../model/yolov5s_relu.onnx rv1106 i8 ../model/yolov5s_relu.rknn
```

运行 RKNN Python Demo：

```
python yolov5.py --model_path ../model/yolov5s_relu.rknn --target rv1106
```

脚本运行成功后，输出信息示例如下。其中 `class` 表示预测类别，`score` 为得分，`(xmin, ymin)` 为检测框左上角坐标，`(xmax, ymax)` 为右下角坐标。

```
# class @ (xmin, ymin, xmax, ymax) score
person @ (209 243 286 510) 0.880
person @ (479 238 560 527) 0.871
person @ (109 238 231 534) 0.840
bus    @ (91 129 555 464) 0.692
```

6. 常见启动问题与下一步

本章只覆盖首次运行中最常见的连接、Server 和内存问题。更多工具安装、模型转换及系统化故障排查，请参考《RKNN2_SDK_开发指南》；接口相关问题请根据使用方式参考《RKNN2_Toolkit_Python_API_参考》《RKNN2_Toolkit_Lite_Python_API_参考》或《RKNN2_Runtime_C_API_参考》。

6.1 设备无法识别

如果执行 `adb devices` 看不到设备，可以依次检查：

1. 检查连线是否正确，重新插拔数据线；
2. 更换计算机 USB 端口；
3. 更换数据线；
4. 使用 USB 连接开发板时，确保本地计算机和 Docker 容器中同时只运行一个 ADB Server。如果需要在 Docker 容器中连接开发板，可先在本地计算机执行：

```
adb kill-server
```

如果出现以下错误，表示系统中未安装 ADB：

```
command 'adb' not found, but can be installed with:
sudo apt install adb
```

执行以下命令安装：

```
sudo apt install adb
```

6.2 `rknn_server` 未正常启动

RKNN-Toolkit2 的连板调试功能要求板端已安装 RKNPU2 环境，并且 `rknn_server` 正常运行。部分开发板系统不会开机自动启动 `rknn_server`，此时运行 RKNN Python Demo 可能出现：

```
E init_runtime: The rknn_server on the concected device is abnormal.
```

可手动启动：

```
adb shell
restart_rknn.sh
```

如果仍然异常，请返回 3.3.2 检查 RKNPU2 Mini Runtime 和 Server 检查组件和版本。

6.3 内存不足导致模型加载失败

RV1103/RV1103B/RV1106/RV1106B 开发板内存较小。如果板端已经运行 `rkipc` 进程，可能导致模型加载失败并出现：

```
E init_runtime: Exception: RKNN init failed. error code: RKNN_ERR_MODEL_INVALID
```

可以进入板端结束 `rkipc` 进程：

```
adb shell

# 查看当前运行的进程，找到 rkipc 对应的 PID
top

# 结束 rkipc 进程
kill -9 PID
```

6.4 后续文档导航

完成首次运行后，请根据后续开发任务进入对应的主文档，避免在快速入门中继续查找完整开发流程、API 说明或高级配置。

检索需求 / 学习目标	推荐参考文档
RK3588 等平台：RKNN2 SDK 首次环境搭建、模型转换与部署验证	《RKNN2_SDK_快速入门》
RK2118 平台：RKNN2 SDK 首次环境搭建、模型转换与部署验证	《RKNN2_RK2118_快速入门》
全流程开发：RKNN2 软件栈架构、模型转换与量化、精度/性能分析、完整部署及故障排查	《RKNN2_SDK_开发指南》
PC 端模型转换与分析：RKNN-Toolkit2 Python API 方法、参数及返回值说明	《RKNN2_Toolkit_Python_API_参考》
板端 Python 轻量部署：RKNN-Toolkit-Lite2 Python API 方法、参数及返回值说明	《RKNN2_Toolkit_Lite_Python_API_参考》
板端 C/C++ 部署：板端 C API、结构体、枚举、错误码及 Mini Runtime 相关接口	《RKNN2_Runtime_C_API_参考》
模型兼容性评估：RKNN 算子支持列表及其约束条件	《RKNN2_算子支持与约束参考》

> **说明：** 本文档面向 RV1103/RV1103B/RV1106/RV1106B，板端运行库统一使用 RKNPU2 Mini Runtime（`librknnmrt.so`）。《RKNN2_Runtime_C_API_参考》为 Runtime C API 的文档名称，阅读时应结合当前平台及 SDK 版本使用对应接口。